

System Design Document: Web Crawl Scheduler

Senior Backend Engineer Design Assignment

Tavily Candidate Assessment

May 21, 2026

Abstract

This document outlines the architecture for a highly scalable web crawler designed to continuously discover and retrieve data from the web. The system is engineered to crawl 1 billion pages per day across 50 million unique domains, intelligently balancing priority, strictly enforced politeness, and freshness while handling downstream failures gracefully.

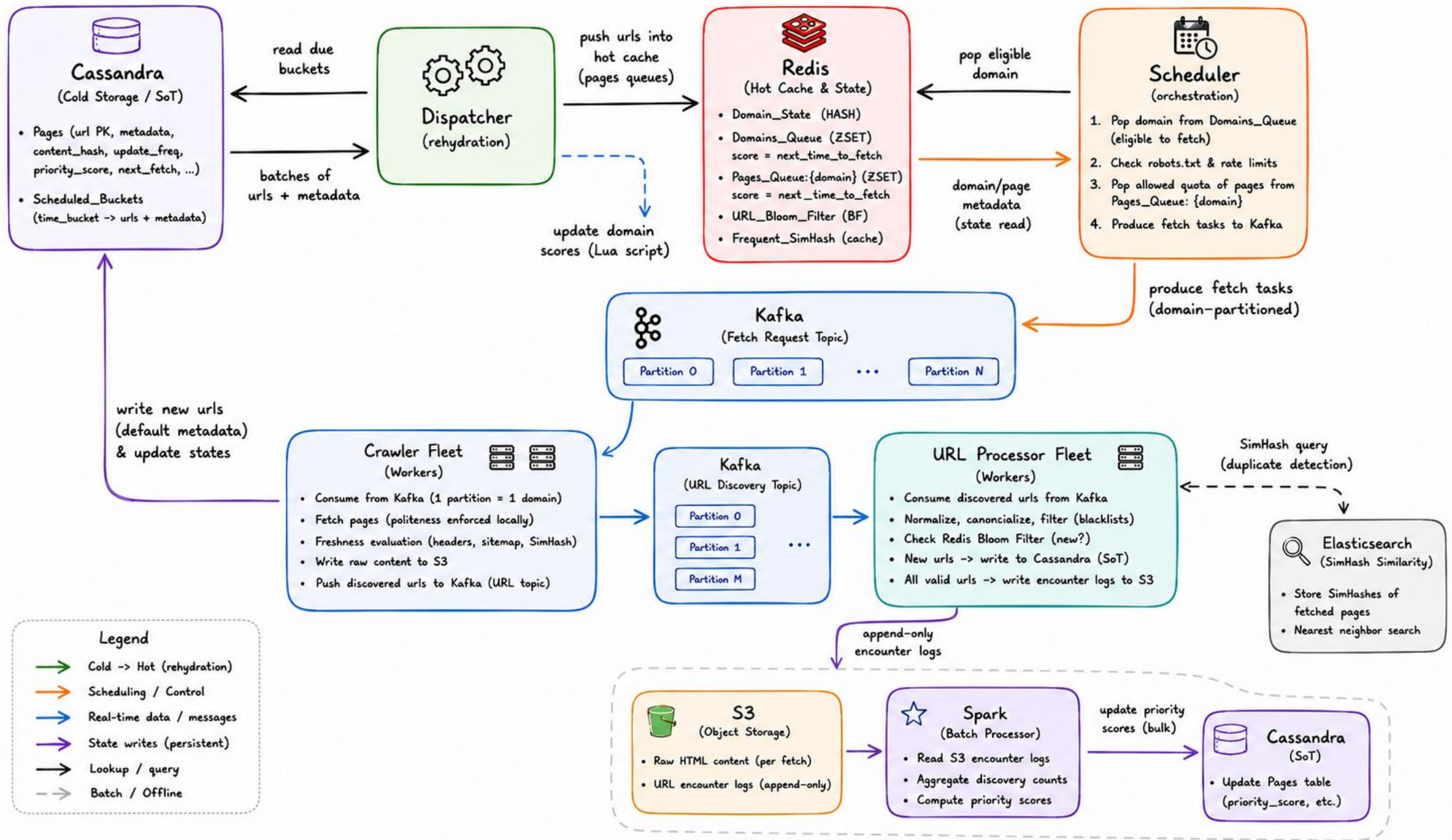
1 Architecture

1.1 Major Components

The architecture separates concerns into horizontal processing tiers linked via message queues to protect core databases from I/O congestion and CPU-heavy operations.

Component	Function
Dispatcher	A background orchestration service managing the transition of data from cold storage to the hot cache. It promotes URLs into the hot cache and ensures domains are queued for the Schedulers.
Scheduler	A service acting as the system's primary valve. It fetches data from the hot cache sorted domains and pages metadata to determine eligible domains, and produces to the crawlers which pages to crawl now.
Crawler Fleet (Workers)	Instances that execute the HTTP requests. They determine content freshness on the fly, update metadata, extract links, produce them to the URL Processors via Kafka, and write raw content to S3.
URL Processor Fleet	Consumers that isolate genuinely new URLs from massive batches of raw links extracted by the crawlers by performing rigorous normalization, blacklist filtering, and deduplication. They also handle the log writing of each URL for the later prioritization calculation and update.
Batch Processor	A background computing cluster utilizing Apache Spark that runs periodic jobs over S3 logs to calculate dynamic URL priorities based on web discovery frequency.

Table 1: Core System Components



1.2 Data Technologies & Storage Layers

Technology	Role	Description
Cassandra	Persistent Storage & SOT	Handles high-throughput writes across two roles: Time-Series Bucketing for future schedules and the authoritative Source of Truth (SoT) for the web graph.
Redis	Hot Cache & State	Provides sub-millisecond, in-memory data structures (Queues, Sorted Sets, Bloom Filters) to strictly govern scheduling constraints and politeness, and caches the most highly repeated SimHashes.
Kafka	Message Queue	Decouples processing tiers into two pipelines: a partitioned Fetch Request Topic, which uses the domain as a key so that pages from the same domain arrive at the same partition, allowing each domain's pages to be fetched by the same crawler to enforce strict rate limiting, and a high-volume URL Discovery Topic.
Amazon S3	Object Storage	Serves as the immutable object store where raw HTML payload data is written immediately after a successful fetch, and stores append-only batches of URL encounter logs for batch processing.
Apache Spark	Batch Engine	Runs asynchronous periodic jobs over the massive S3 URL encounter logs to recalculate dynamic URL priority scores.
Elasticsearch	Search Engine	Serves as the cold tier for querying SimHash similarities to detect cross-URL duplicate content globally.

Table 2: Storage Layer Infrastructure

1.3 Data Flow & Interactions

- **Dispatch & Rehydration:** The Dispatcher continuously sweeps the current Cassandra time-bucket, reading batches of URLs whose scheduled fetch times have arrived. It pushes the individual pages and their metadata payload into their respective Redis Page Sorted Sets according to their `next_time_to_fetch`. To update the Domains Queue without incurring network read delays, it fires a pipelined Lua script to Redis, atomically setting the domain's score to: `max(next_time_to_fetch, next_time_allowed_to_fetch)`
- **Scheduling & Queuing:** Scaled Schedulers pop domains from the Redis Domains Queue based on the domain's `next_time_to_fetch` (where the score is $<$ the current time). The Scheduler verifies the domain's cached robots.txt, then pops the allowed quota of pages from the domain's individual Pages Queue according to the pages' `next_time_to_fetch`. It produces them to Kafka, updates the domain's state hash, and resets its queue score to the next future limit, putting the domain back to sleep.

- **Fetching & Freshness Evaluation:** Crawler workers consume crawl requests from the strictly partitioned Kafka topic.
 - **Parallel Batch Fetching:** The crawlers first retrieve the raw HTTP data, executing these fetches in heavy batches using parallel requests to maximize network throughput while localizing domain rate limits in-memory.
 - **Freshness Validation:** Once the data is retrieved, crawlers validate its freshness. They first perform lightweight validation using sitemaps and HTTP headers (e.g., Last-Modified). If these are inconclusive, they calculate a new 64-bit SimHash on the fetched page content and compare it directly against the historical content hash passed alongside the record in the Kafka message.
 - **First-Fetch Duplicate Checks:** For pages flagged as a first-time fetch (`is_first_fetch`), the calculated SimHash is queried against the hot SimHash cache and Elasticsearch to identify global duplicates.
 - **Downstream Pipeline Execution:** After freshness is determined, two actions are triggered:
 1. **Schedule & State Update:** The crawler dynamically recalculates the page's `update_frequency` and `next_time_to_fetch`. It then updates the Cassandra Source of Truth (SoT) and writes the metadata to the corresponding future Cassandra time-bucket based on the newly calculated `next_time_to_fetch`.
 2. **Link Extraction & Discovery:** If the page content is confirmed to be fresh/updated (or if it is a newly crawled page), the crawler parses and extracts all outgoing URLs from the HTML and publishes them to the Kafka URL Discovery Topic for downstream processing.
- **URL Discovery:** URL Processors act as the system's primary garbage collectors. They consume heavy batches of raw links from Kafka and immediately execute a strict sanitization pipeline: URL canonization, normalization, and fast redirect resolution. Next, they query these cleaned URLs against known bad domain lists. For the surviving URLs, processors issue pipelined queries against the Redis Bloom Filter. Only the genuinely new URLs are written directly to Cassandra with default baselines. Another role of the URL Processors is taking every valid URL they encounter (both new and previously seen) and writing them in heavy batches to S3 encounter logs.
- **Priority Calculation (Batch Job):** A scheduled Apache Spark job periodically reads the URL encounter logs from S3. It aggregates the total number of times each URL has been discovered across the web, calculates a new dynamic priority score, and updates the Cassandra Source of Truth in bulk.

2 Core System Assumptions

Before detailing the data model and scheduling logic, the system's architecture relies on several foundational assumptions regarding the web graph and infrastructure constraints:

- **System Maturity (High Re-crawl Ratio):** We assume the crawler has been operating long enough that it possesses a massive seed list. Consequently, the vast majority of daily operations are re-fetches of known pages, rather than the initial discovery and onboarding of completely unseen domains.
- **Dual-Factor Scheduling:** The optimal time to fetch a page is not strictly tied to its update rate, but the longer a page hasn't been fetched while updated, the higher its urgency to be

fetches. We assume fetch urgency is a product of both business priority and the historical metrics we track regarding its expected next update time.

- **High Duplication Rates:** A significant percentage of discovered URLs (even post-normalization) point to identical content (e.g., standard boilerplate, parked domains, mirrored sites). We assume aggressive duplicate pattern tracking and content hashing are mandatory to avoid crippling the system with redundant fetches.
- **Workload Distribution via Domain Partitioning:** We assume that randomly partitioning domains across the crawler fleet (e.g., via Kafka domain hashing) will result in an equal distribution of fetched pages among workers. This relies on the statistical assumption that both “heavy” domains (with millions of pages) and “light” domains will be distributed evenly across partitions, ensuring that each worker handles approximately the same amount of pages per second without causing massive hot spots.
- **Network & DNS Feasibility:** We assume DNS resolution is handled efficiently via local caching mechanisms on the worker nodes, preventing DNS lookups from becoming the primary bottleneck for the Crawlers.
- **Average Payload Size:** We assume an average raw HTML payload size of ~100 KB per page. This establishes a predictable baseline for S3 storage growth (~100 TB/day) and network bandwidth provisioning.

3 Data Model

3.1 Cassandra Schemas (Cold Storage)

Table	Keys	Schema
Pages (SoT)	url (PK)	domain, content_hash, last_fetched_time, next_time_to_fetch, update_freq, priority, state, is_first_fetch.
Scheduled Buckets	time_bucket_minute (Partition)	domain, content_hash, priority, update_freq, next_time_to_fetch, is_first_fetch.
	url (Clustering)	<i>(Note: Duplicating metadata here allows the Dispatcher to populate Redis without expensive secondary lookups).</i>

Table 3: Cassandra Storage Schemas.

3.2 Redis Schemas (Hot State)

To optimize in-memory performance and maintain a highly structured memory footprint, the Redis tier is partitioned into two functional categories: queue management (ZSETs) and state tracking/caching.

Structure	Key	Score	Data
Domains_Queue (ZSET)	domains_queue	next_time_to_fetch (<i>Lua-clamped</i>)	domain_name
Pages_Queue (ZSET)	pages_queue:{domain}	next_time_to_fetch	Serialized JSON payload containing: url, content_hash, priority, update_freq, is_first_fetch.

Table 4: Redis Sorted Sets (Queue Structures)

Structure	Key	Details
Domain_State (Hash)	{domain}	Stores domain-level throttling metrics: current_rate_limit, next_time_allowed_to_fetch, failures_last_5m, and circuit_breaker_cooldown_until.
URL_Bloom_Filter	global_url_bloom	A global, probabilistic bit-array optimized for low-latency, memory-efficient lookup to filter out previously seen URLs.
Frequent_SimHash	frequent_simhash	Caches the most commonly repeated SimHashes across the web to prevent hitting Elasticsearch for known boilerplate or duplicate pages.

Table 5: Redis State & Cache Structures

4 Scheduling Logic

4.1 Politeness: Multi-Tier Enforcement

- **The Scheduler (Soft Limitation & Quota Management):** To prevent Kafka from being congested with URLs from a single domain that cannot actually be fetched concurrently, the Scheduler acts as a preliminary valve. When popping a domain from the Redis `Domains_Queue`, it consults the cached `robots.txt` rate limits. It only pulls an “allowed quota” of URLs from the `Pages_Queue` for that specific time window.
- **Kafka Partitioning (Traffic Routing):** When the Scheduler produces fetch commands to

the Kafka Fetch Request Topic, it uses the domain as the partition key. This foundational architectural decision guarantees that every single URL belonging to a specific domain is consistently routed to the exact same Kafka partition.

- **Crawler Workers (Strict In-Memory Enforcement):** Because Kafka ensures a single partition is consumed by a single Crawler instance, all traffic for a given domain is inherently isolated to one machine. This allows the Crawler to manage the strict, final enforcement of rate limits (e.g., executing the exact crawl-delay sleep intervals) entirely within its local memory.
- **Worker Politeness Overflow (Safety Valve):** If, despite all prior scheduling filters, a crawler worker receives a page that it cannot fetch without violating the domain's strict rate limits (for example, due to a sudden drop in domain responsiveness or transient scheduling spikes), the worker will not block its execution thread. Instead, it immediately re-routes the page back to the Cassandra `Scheduled_Buckets` (cold tier) with an applied exponential backoff penalty, freeing the worker to handle other eligible traffic.

4.2 Reliability & Failure Handling

- **Status Triage (Permanent Errors):** Permanent errors (e.g., 404 Not Found) trigger a strict penalty, pushing the URL's `next_time_to_fetch` far into the future, rather than completely flagging it as permanently dead.
- **Single Page Penalization (Transient Errors):** For individual page failures (e.g., 5xx Server Errors, timeouts), the system "punishes" the URL by applying an exponential backoff to its `next_time_to_fetch`. This pushes the failing page further back in the scheduling queue, ensuring we do not waste cycles repeatedly hammering a broken link.
- **Distributed Circuit Breaker:** A dedicated Monitor Service continuously tracks crawler telemetry, aggregating failures per domain over a rolling sliding window. If a domain's transient failures exceed a specified threshold percentage, the Monitor Service trips a Redis circuit breaker. It actively updates the cached `Domain_State` and pushes the domain's `next_time_allowed_to_fetch` into the future by a designated cooling-off period.
- **Observability-Driven Auto-Scaling:** To maintain system equilibrium, a dedicated monitoring layer continuously tracks Kafka topic lag (queue depth) and Redis Sorted Set memory consumption. These telemetry metrics directly drive our infrastructure's scaling policies, allowing the system to dynamically increase or decrease the active instances in the Crawler, Scheduler, and URL Processor fleets to absorb traffic spikes and prevent memory exhaustion.

4.3 Freshness

To track content changes efficiently without wasting resources, we evaluate the raw update rate independently:

- **Adaptive Polling:** Every page maintains an `update_frequency` metric, representing the expected interval between content changes. Instead of a static guess, this metric is dynamically recalibrated by the Crawler after every fetch based on content change rates.
- **Penalty for Stale Hits:** If a page is fetched but the content hasn't changed (based on the SimHash comparison), we exponentially decrease the frequency attribute (i.e., we increase the wait time interval). This prevents the system from wasting resources polling dead or static pages.

- **Reward for Fresh Hits:** If we find that the content did update before its expected `next_update_time`, we adjust the `update_frequency` attribute to a lower value, ensuring we poll it more aggressively in the future.
- **Early Fetching:** A page can be fetched before its `next_update_time` based on its high priority, or if the overall system is in a healthy condition and possesses surplus resources to fetch pages ahead of their evaluated schedule.

4.4 Prioritization

Business urgency and system-level penalties act as modifiers against the raw freshness:

- **Priority (Dynamic Calculation):** Priority is a dynamic metric continuously refined by our batch processing layer. By aggregating the S3 URL encounter logs, Spark calculates the discovery frequency of every known URL. URLs that are discovered more frequently across the web are automatically assigned higher priority scores, increasing their fetch urgency.
- **Near-Duplicate Penalization:** When a URL is fetched for the first time (`is_first_fetch` flag), its computed SimHash is compared against the global pool of known hashes to detect cross-URL duplicates. If the page is deemed a duplicate of an existing known page, it is immediately marked as such, and its `update_frequency` and `priority` score are drastically lowered to ensure we rarely waste resources fetching it again.

4.5 The Scheduling Formula

Combining Freshness and Priority: To determine the definitive order of execution, the Crawler calculates a linear combination of the adaptive freshness and the priority score:

$$\text{next_time_to_fetch} = \text{CurrentTime} + \frac{\text{update_frequency}}{\text{weight} \times \text{priority}}$$

(Where *weight* is a tunable hyperparameter).

5 Technology Choices & Design Rationale

5.1 Technology Choices

- **Cassandra:** Chosen primarily for its write-heavy workload capabilities. It effortlessly handles massive, continuous writes, and its architecture means updating a record requires no prior reading (upserts). We leverage its native time-series bucketing to efficiently organize and feed the Redis Sorted Sets with upcoming crawl schedules.
- **Redis:** Selected for its sub-millisecond latency. We utilize a variety of its data structures (Sorted Sets, Bloom Filters, and Hashes) to manage state and queues. Crucially, we actively and strictly manage its RAM consumption through our cold-to-hot tiering pipeline to keep the cluster healthy, manageable, and cost-effective.
- **Kafka:** Essential for its massive scaling abilities. Its native partitioning feature is the exact mechanism that guarantees all traffic for a specific domain routes to a single crawler, empowering the crawlers to strictly enforce rate limits locally.
- **Elasticsearch:** Chosen specifically for its native capabilities to execute low-latency Hamming distance vector searches, which is perfectly suited for our global SimHash duplicate detection.

- **Apache Spark:** Selected to handle the massive volume of data processing required for dynamic prioritization. It efficiently executes map-reduce jobs over billions of S3 log entries to compute URL priority in bulk.
- **Languages:** C++ is utilized for the Crawler and URL Processor fleets because they handle massive, CPU-intensive HTML parsing, hashing, and sanitization jobs. Python is used for the Dispatcher and Scheduler services, as their orchestration workloads are heavily I/O-bound rather than CPU-bound.

5.2 Design Rationale

- **Cold-to-Hot Storage Tiering (Cassandra Bucketing):** Storing the metadata for billions of pages permanently in Redis is financially and technically unfeasible due to RAM constraints. Furthermore, it is unnecessary: the priority queues only care about pages scheduled in the immediate future. By bucketing future crawl schedules in Cassandra based on their `next_time_to_fetch`, we create an inexpensive cold-storage pipeline. The Dispatcher then promotes only the immediately relevant buckets into Redis. This architecture exploits Redis's high efficiency for priority queuing without bloating it with dormant data.
- **Two-Tier Queuing (Preventing Head-of-Line Blocking):** A naive approach might place all URLs into a single, massive global Pages Sorted Set. However, if 2,000 Wikipedia pages suddenly become urgent, but Wikipedia's crawl-delay strictly limits us to 20 requests per second, a single queue would force the Scheduler to waste cycles paginating, repeatedly validating limits, and skipping blocked URLs. By separating the queue into two tiers—a `Domains_Queue` followed by individual `Pages_Queue: {domain}`—we bypass this entirely. The Scheduler first finds domains that are ready and legally permitted to be fetched, and only then queries those specific domains for their top-priority pages.
- **The “Two Clocks” Resolution (Atomic Lua Evaluation):** For each batch of pages the Dispatcher writes to the pages queue, it must update the scheduling state of the domains those pages belong to. Specifically, it must reconcile the domain's overall `next_time_to_fetch` with its politeness constraint (`next_allowed_fetch_time`). Resolving these conflicting clocks traditionally creates severe network thrashing and excessive round-trips for each batch. We solve this by using an atomic Redis Lua script that takes both parameters into consideration server-side, evaluating the maximum constraint and updating the domain's score atomically.
- **SimHash State Pass-Through (Historical Content Check):** Caching 64-bit SimHashes for 1 trillion known URLs in Redis to check if content changed would require upwards of 130 TB of RAM. Instead of querying a database or cache for each refetched page, the system adopts a state pass-through pattern. Required metadata (including the old content hash) is duplicated in Cassandra buckets, passed directly through the Redis queue member as a JSON string, and injected into Kafka. The crawler reads it directly from the Kafka record. For every refetched page, this zero-read-latency approach allows the crawler to instantly compare the newly calculated hash against the previous one, entirely eliminating expensive database queries during the core fetch loop.
- **Cross-URL Deduplication (Global Similarity Check):** For newly discovered URLs, we must check if their pages are duplicates of other URLs. We use Redis as a fast cache for highly repeated pages with different URLs, and Elasticsearch as the second layer that keeps SimHashes for all fetched pages, executing the complex approximate nearest neighbor queries required for duplicate detection.

- **Overcoming the URL Deduplication Resource Barrier (Bloom Filter vs DB):** The naive approach to URL deduplication involves checking every newly discovered URL against a database of known URLs. At a scale of 1 billion pages per day (~11,500 pages/sec), assuming 100 links per page, the system must validate over 1.1 million URLs per second (with peaks reaching several millions). Conventional databases like Cassandra cannot efficiently sustain this read-throughput for deduplication.

While Redis could handle the throughput, the memory cost is prohibitive: keeping even a 128-bit hash per URL for 100 billion unique URLs (post-canonization) equates to roughly 1.6 Terabytes of RAM. Using a Bloom Filter drastically reduces this footprint. Configuring a filter for 100 billion items with a 0.01% false-positive rate requires “only” ~160 GB of memory.

We must acknowledge the tradeoff: a 0.01% (or 0.1%) false-positive rate means we will occasionally flag a genuinely new URL as “seen,” causing it to be discarded. However, this is an acceptable tradeoff for a web crawler; if a page is important, we will likely discover multiple distinct links pointing to it across the web over time, exponentially decreasing the probability that the system permanently misses the underlying content.

- **Dynamic Prioritization via Batch Processing:** URL priority is generally not a fast-changing metric. Maintaining real-time distributed counters for all discovered URLs is extremely resource-heavy and introduces massive write-lock contention. It is much more efficient to offload this to an asynchronous Spark batch job reading append-only S3 logs, computing and updating priority scores in bulk once in a while without impacting the critical real-time discovery pipeline.

6 Possible Extensions

The current architecture can be extended with additional scheduling lanes for heterogeneous workloads with minimal architectural changes. For example, pages with very high update frequency could be routed through a dedicated fast lane with higher resource allocation, while low-frequency pages remain on the default path. Similarly, large-content pages could be separated into a slower lane with longer fetch timeouts and lower Kafka emission rates. Because the system already separates workloads through Kafka topics, Redis queues, and independent crawler fleets, this extension would mainly require introducing additional scheduling policies and dedicated processing paths rather than a fundamental redesign of the architecture.